



tecnun

CAMPUS TECNOLÓGICO DE LA UNIVERSIDAD DE NAVARRA
NAFARROAKO UNIBERTSITATEKO CAMPUS TEKNOLOGIKOA
Escuela Superior de Ingenieros • Ingeniarien Goi Mailako Eskola

Examen Diciembre C++



Informática II

Prof. Paul Bustamante / Jesus Paredes

Nombre:

Carné:

Índice

.....	1
1. INTRODUCCIÓN.....	1
2. EJERCICIO 1: BUFFER CIRCULAR (2.5 PTS.).....	1
3. EJERCICIO 2: GESTIÓN DE TALLER CON CLASES (2.0 PTS.).....	3
4. EJERCICIO 3: HERENCIA, POLIMORFISMO Y FICHEROS CON POO (3.0 PTS.).....	5

1. Introducción

Recuerde que debe dejar los ejercicios en la carpeta “**C:\Temp\ExamInfor**”. Si no los deja allí, no se le podrá corregir el examen. Trate de realizar primero el ejercicio que crea que es más sencillo y deje el complicado para el final.

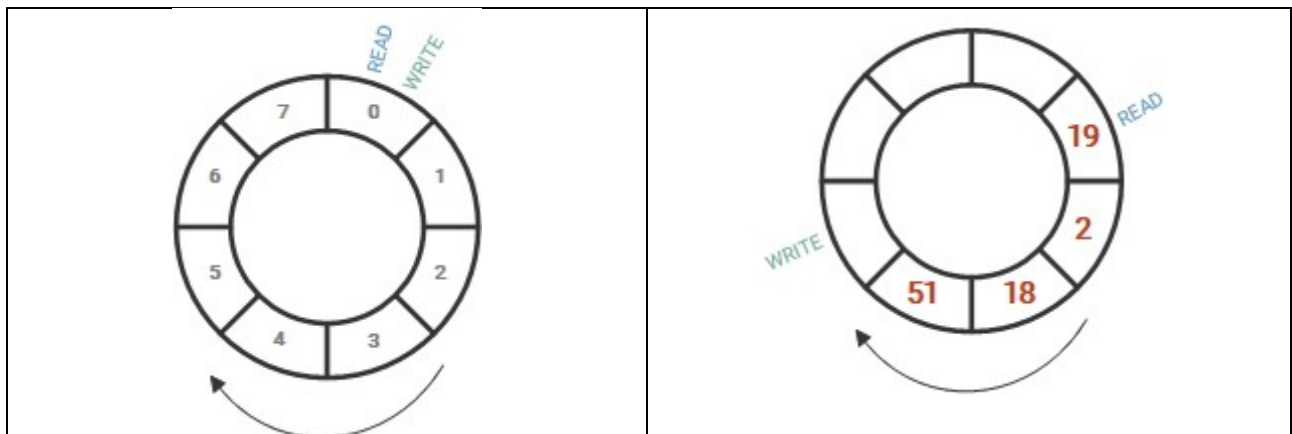
2. Ejercicio 1: Buffer Circular (2.5 Pts.)

A Ud. le han pedido en su empresa que haga un programa que permita crear un buffer circular.

En la siguiente figura se puede ver una representación visual de lo que es un **array** o vector de números enteros:



En la siguiente figura se puede ver una representación visual de lo que es el mismo **array** o vector como buffer circular, es decir que el inicio está conectado con el final del vector de tal forma que puedo ir introduciendo datos y sacando datos, usándolo como FIFO (**First Input First Output**), es decir el primer número que introduzco será el primero que saco:



Para la implementación del buffer circular deberá tener en usar dos índices (**read** y **write** en la figura), de tal forma que cada vez que introduzco un dato, el índice **write** aumenta (se aleja del índice **read**) y cuando saco un dato, el índice **read** aumenta (se acerca al índice **write**). Cuando el índice **write** llega al final del vector, deberá pasar a la posición “0”. Lo mismo con el índice **read**.

Programa:

Al inicio del programa deberá pedir el **tamaño del buffer** (sólo una vez) para crearlo de forma dinámica. Luego ya podrá trabajar con ese vector.

```

Select C:\Users\pbustamante\OneDrive - Tec...

** CIRCULAR BUFFER **
Give the size of the buffer:?5

** CIRCULAR BUFFER **
1  Put Value to buffer
2  Get Value from buffer
3  Check status of buffer
4  Exit Program
   Opc:3

```

Luego el programa deberá mostrar las siguientes opciones:

- **Opción 1:** Agregar un dato al buffer: en este ejemplo he agregado el 100 y 200. Si el buffer se llena, deberá mostrar un mensaje de “**buffer lleno**” (figura derecha)

```

C:\Users\pbustamante\OneDrive - Tec...

** CIRCULAR BUFFER **
1  Put Value to buffer
2  Get Value from buffer
3  Check status of buffer
4  Exit Program
   Opc:1

Put a number to store in the buffer:?100

** CIRCULAR BUFFER **
1  Put Value to buffer
2  Get Value from buffer
3  Check status of buffer
4  Exit Program
   Opc:1

Put a number to store in the buffer:?200

** CIRCULAR BUFFER **
1  Put Value to buffer

```

```

C:\Users\pbustamante\OneDrive - Tec...

** CIRCULAR BUFFER **
1  Put Value to buffer
2  Get Value from buffer
3  Check status of buffer
4  Exit Program
   Opc:1

Buffer is full!

** CIRCULAR BUFFER **
1  Put Value to buffer
2  Get Value from buffer

```

- **Opción 2:** Sacar un dato del buffer: en este caso, el primer dato que saca es el 100 (que fue el primer dato introducido). Si no hay más datos, deberá mostrar un mensaje de “**buffer vacío**”.

```

C:\Users\pbustamante\OneDrive - Tec...

** CIRCULAR BUFFER **
1  Put Value to buffer
2  Get Value from buffer
3  Check status of buffer
4  Exit Program
   Opc:2

The output value is 100

** CIRCULAR BUFFER **
1  Put Value to buffer
2  Get Value from buffer
3  Check status of buffer

```

```

C:\Users\pbustamante\OneDrive - Tec...

** CIRCULAR BUFFER **
1  Put Value to buffer
2  Get Value from buffer
3  Check status of buffer
4  Exit Program
   Opc:2

Buffer is empty!

** CIRCULAR BUFFER **
1  Put Value to buffer
2  Get Value from buffer
3  Check status of buffer

```

- **Opción 3:** mostrar el estado del buffer: Si está lleno, vacío o el número de elementos que hay en él:

```

C:\Users\pbustamante\OneDrive - Tec... - [X]
** CIRCULAR BUFFER **
1 Put Value to buffer
2 Get Value from buffer
3 Check status of buffer
4 Exit Program
   Opc:23
   There are 2 numbers in the buffer

** CIRCULAR BUFFER **
1 Put Value to buffer
2 Get Value from buffer
3 Check status of buffer

C:\Users\pbustamante\OneDrive - Tec... - [X]
** CIRCULAR BUFFER **
1 Put Value to buffer
2 Get Value from buffer
3 Check status of buffer
4 Exit Program
   Opc:23
   Buffer is empty!

** CIRCULAR BUFFER **
1 Put Value to buffer
2 Get Value from buffer
3 Check status of buffer

```

Requisitos del Programa:

- (1.0 Pts.) Debe hacer la función para introducir un dato `int push_buffer(argumentos)`, la cual devolverá un número (0: si pudo meter el dato en el buffer, 1: si NO pudo porque el buffer está lleno). En los argumentos deberá elegir qué argumentos le debe pasar a la función: el vector, el dato, el índice read y write, etc.
- (1.0 Pts.) Deberá hacer la función para sacar un dato `int pop_buffer(argumentos)`, la cual devolverá el dato en el buffer según el método FIFO. En los argumentos deberá elegir qué argumentos le debe pasar a la función: el vector, el índice read y write, etc. Si el buffer está vacío, no debería llamar a la función y mostrar el mensaje “buffer vacío”.
- Todos los vectores deberán ser dinámicos (con reserva de memoria, usan `new`). Desde estas funciones no debe imprimir el resultado. Esto lo hará en `main`, después de haber llamado a las mismas. **Main** y optimización de código (0.5 pts.) (sino realiza ninguna función, no habrá puntuación en el ejercicio).

3. Ejercicio 2: Gestión de taller con clases (2.0 Pts.)

A Ud. le han pedido que haga un programa para la gestión de un taller de reparación y mantenimiento de coches, usando la POO. Dicho programa deberá tener las siguientes opciones:

- **Opcion1 (0.5 Pts.):** Agregar coche: deberá pedir los datos de matrícula, la hora de ingreso y el número de servicios que se hará (para cada servicio debe pedir la descripción y el precio). Ver Figura 1.
- **Opcion2 (0.8 Pts.):** Pagar servicio: deberá pedir la matrícula del coche (si esta no existe, debe sacar un mensaje). También deberá pedir la hora de finalización (para el coste de la mano de obra). Finalmente deberá sacar detallado el costo total del servicio (materiales más mano de obra). Ver **Figura 3** . Al final, cuando un coche paga, hay que **borrarlo de la lista**.
- **Opcion3 (0.5 Pts.):** Listado de coches en servicio. Ver **Error! Reference source not found.**Figura 3.
- **Opcion4:** Salir.

```

C:\Users\pbustamante\OneDrive - ...
** Taller Coches **
1.Agregar coche
2.Pagar
3.Mostrar lista
4.Salir
    Opc: ?1
Dar matricula:9974abc
Dar Hora entrada (h:m):10 20
Dar num servicios:2
    Descripcion: ?cambio aceite
    Precio: ?150
    Descripcion: ?filtro aire
    Precio: ?50
** Taller Coches **
1.Agregar coche
2.Pagar
3.Mostrar lista
4.Salir
    Opc: ?

```

Figura 1 Agregar coche

```

C:\Users\pbustamante\OneDrive - Tecnun\Tec...
** Taller Coches **
1.Agregar coche
2.Pagar
3.Mostrar lista
4.Salir
    Opc: ?3
-----
Matricula:9974abc
Dar Hora entrada:10:20
Num servicios:2
    cambio aceite      150
    filtro aire        50
-----
Matricula:1234abc
Dar Hora entrada:12:0
Num servicios:2
    cambio escobillas   50
    llenar liquidos     20
** Taller Coches **
1.Agregar coche
2.Pagar
3.Mostrar lista
4.Salir
    Opc: ?

```

Figura 2 Listado de coches

```

C:\Users\pbustamante\OneDrive - Tecnun\Tecnun\Info...
** Taller Coches **
1.Agregar coche
2.Pagar
3.Mostrar lista
4.Salir
    Opc: ?2
Dar matricula: ?9974abc
Dar Hora fin (h:m): ?12 40
-----
Matricula:9974abc
Dar Hora entrada:10:20
Num servicios:2
    cambio aceite      150
    filtro aire        50
Horas o Fraccion:3
Costo x Hora:40
Costo Materiales:200 Euros
Costo Mano Obra:120 Euros
Costo Total:320 Euros
-----
** Taller Coches **

```

Figura 3 Pago del servicio

Programa:

Para la realización del programa, deberá crear las siguientes clases (con las variables y **SIN** cambiar el especificador de acceso, público o privado). Podrá poner los constructores que necesite y agregar más funciones.

<pre> class servicio { char descripcion[30]; double precio; public: servicio(); //agregar más funciones }; </pre>	<pre> class mantenimiento { static double costo_hora; //hora o fracción char matricula[20]; servicio lstServ[10]; //máximo 10 servicios int numServ=0; //número de servicios Time tini, tfin; public: </pre>
---	---

<pre> class Time { public: int hr, min; public: Time(int h=0, int m=0); //agregar mas funciones }; </pre>	<pre> mantenimiento(); //constructor //agregar las funciones necesarias }; </pre>
---	---

En cuanto a **main**, a continuación, se da un esqueleto de cómo podría ser:

```

#define MAX 100
double mantenimiento::costo_hora = 40.0;
void main()
{
    int ns = 0;
    mantenimiento *lista[MAX];

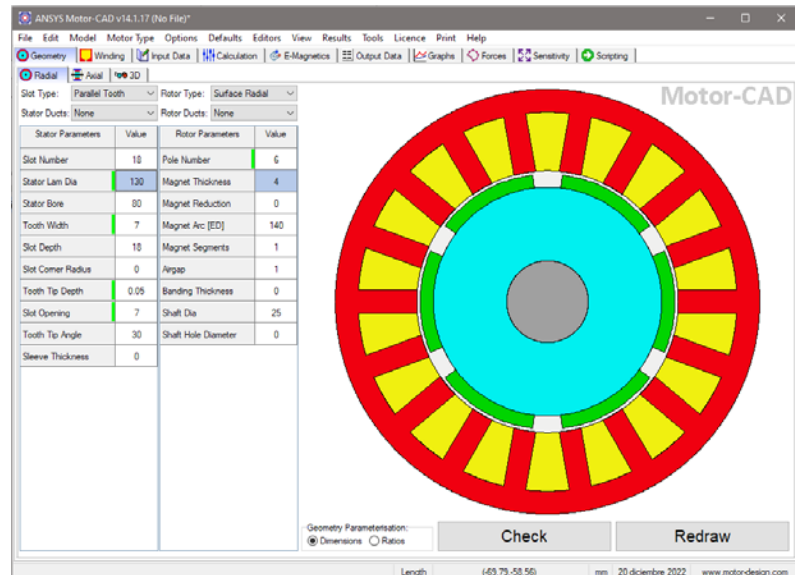
    int opc=0;
    while (1) {
        // mostrar el menú y hacer el resto del código
    }
}

```

Función principal (**main**) más optimización del código: **0.2 Pts.**

4. Ejercicio 3: Herencia, Polimorfismo y Ficheros con POO (3.0 Pts.)

Los distintos softwares de elementos finitos utilizan clases para las distintas etapas del cálculo. En este caso se presenta una parte de un software comercial llamado MotorCAD para el cálculo de máquinas eléctricas. Al igual que otros softwares de elementos finitos las distintas partes del proceso de cálculo se agrupan en distintos bloques. En el caso de MotorCAD se agrupan en distintas pestañas tal como se puede ver en la imagen: geometría, devanado, variables eléctricas...



El proceso de creación de un software de este tipo está basado en clases y herencia. Y como normalmente se desarrollan utilizando versiones hacen uso del polimorfismo. Para demostrar sus conocimientos en esta materia deberá desarrollar una parte muy simplificada de dicho software centrándose en la parte de la geometría. Para ello deberá programar las siguientes clases:

Una **clase base “machine”** sobre la que se construyen el resto de clases. Esta clase tiene variables independiente y variables dependientes (es decir, que se calcula a partir de las variables independientes).

Las variables independientes que se definen como **protected** son:

```
double D_ext_s;    // Diámetro exterior del estator
double w_t_s;      // Espesor del diente del estator
double h_agap      // Altura de entrehierro (espacio separación entre rotor y estator)
double h_slot;     // Altura de ranura
int Q_s;           // Número de ranuras de estator (parte en amarillo)
```

La variable independiente definida como **public** es:

```
double D_int_s;    // Diámetro interior del estator
```

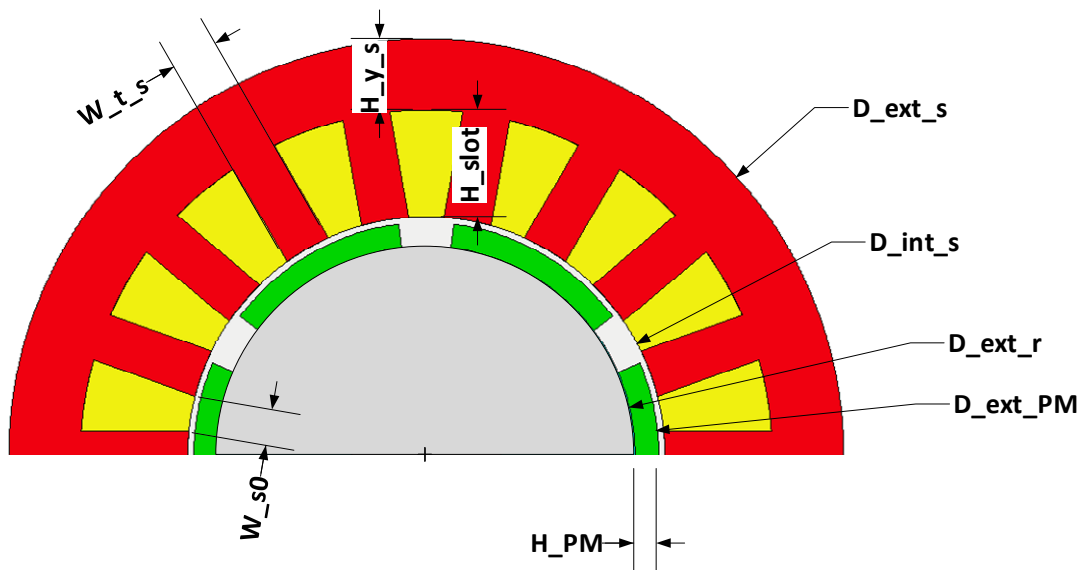
Las variables dependientes definidas como **private** son las siguientes:

```
double D_ext_r;    // Diámetro exterior del rotor
double w_s0;       // Anchura de la apertura de la ranura
double h_y_s;      // Altura del yugo del estator
```

La clase “SPM” (Surface Permanent Magnet) es una clase que **hereda** de la clase “**machine**” todas sus variable y funciones de forma **pública**. Esta clase define un tipo específico de rotor (parte giratoria), en este caso de imanes superficiales.

Todas las variables de esta clase son **independientes** y definidas como **privadas**:

```
double h_PM;       // Altura de los imanes
double D_ext_PM;    // Diámetro exterior de los imanes
```



- **Opción 1 (1.25 Pts.):** Cree los constructores y destructores que permitan **inicializar** las **variables independientes** desde el main para la clase **machine** y para la clase derivada **SPM**. Dentro de los constructores no se podrá incluir sentencias “cin” “cout” para pedirle al usuario los datos.
 - **Dentro de los constructores** deberá llamar a la función que calcula las variables independientes. Esta función deberá definirse para la clase base y para las clases derivadas definida como:

```
void f_dependent_vars_machine() // Función de la clase base
void f_dependent_vars_SPM()    // Función de la clase derivada
```


- Las ecuaciones que debe aplicar dentro de esta función (**f_dependent_vars_machine()**) para la clase **machine** son las siguientes:

$$w_{s0} = \left| D_{int_s} \cdot \sin \left(\frac{\pi}{Q_s} - \text{asin} \left(\frac{w_{ts}}{D_{int_s}} \right) \right) \right|$$

$$h_{ys} = \frac{D_{ext_s}}{2} - \frac{D_{int_s}}{2} - h_{slot}$$

$$D_{ext_r} = D_{int_s} - 2 * h_{agap}$$

- Las ecuaciones que debe aplicar dentro de esta función (**f_dependent_vars_SPM()**) para la clase **SPM** son, **además** de las de la clase **machine**, las siguientes:

$$D_{ext_{PM}} = D_{int_s} - 2 * h_{agap}$$

$$D_{ext_r} = D_{ext_{PM}} - 2 * h_{PM}$$

- Ponga en un comentario en qué cambiaría el código si las funciones **f_dependent_vars_machine()** y **f_dependent_vars_SPM()** tuviesen (ambas) el mismo nombre.
 - Complete el código del **main** para crear un **nuevo** objeto de la lista pidiéndole los datos al usuario a través de la consola.
- **Opción 2 (0.75 Pts.):** Cree una función que muestre por pantalla los datos de las máquinas para la clase **machine** y para la clase derivada **SPM**. En ambos casos deberá tener el mismo nombre. Complete el código del **main** para mostrar por pantalla **todos** los datos de los objetos de la lista

void prt()

- **Opción 3 (0.75 Pts.):** Cree una función perteneciente a la clase **machine** (solo en esta clase) que permita guardar las variables en un fichero de texto. Para ello se le pasará como variable de entrada un objeto de la clase **ofstream** (correspondiente al fichero de escritura), tal como se indica a continuación. Complete el código del **main** para guardar todas las máquinas en un archivo "Datos.txt"

void Escribir_fichero(ofstream&)

- **Opción 4 (0.25 Pts.):** Haga lo mismo que en la opción anterior, pero en lugar de para escribir en un fichero, para leer desde un fichero una máquina. La función no tiene variables de salida. Como variable de entrada deberá tener un objeto de tipo **ifstream**, y no debe devolver ningún tipo de variable. Complete el código del **main** para cargar todas las máquinas desde un archivo "Datos.txt"

<pre> class machine { private: double w_s0; double h_y_s; double D_ext_r; protected: double D_ext_s; double h_agap; int Q_s; double h_slot; double w_t_s; public: double D_int_s; public: //constructores & destructores machine(...); void f_dependent_vars_machine(); void prt() void Escribir_fichero(ofstream&); void Cargar(istream&); }; </pre>	<pre> class SPM : public machine { private: double D_ext_PM; double h_PM; public: //constructores & destructores SPM(...); void f_dependent_vars_SPM(); void prt() }; </pre>
<pre> void main() { machine *lista[100]; lista[0] = new machine(130,80,1,36,18, 7); lista[1] = new machine(120, 80, 1, 36, 18, 5); lista[2] = new SPM(150, 80, 1, 36, 18, 5, 10); int cont=3; // variable contador de elementos // Ponga un menú para que el usuario pida los datos y crear nuevos objeto // // Menu con opciones: 1.Agregar 2.Listado 3.Grabar a fichero 4.Leer de fichero 5.Salir } </pre>	

¡Suerte!!